

What is the DOM?

- The **DOM** (**Document Object Model**) is a tree-like structure that the browser creates from your **HTML**.
- It represents all the elements on the page such as headings, buttons, divs and everything becomes a node in this tree.
- **JavaScript** can interact with this DOM to update content, change styles, or handle user actions.

What is the Real DOM in React?

- In a React project, the **Real DOM** is the actual HTML page your browser reads.
- It lives inside the **index.html** file in the public folder.
- There's usually a **<div id="root">**
</div> inside that file. That's the main entry point of your app.
- React grabs this root element and injects your app into it.

What is the Virtual DOM?

- The **Virtual DOM** is a lightweight JavaScript copy of the Real DOM.
- It's created using **JSX** inside files like **App.js** or other component files.
- React uses this virtual version to track changes without directly touching the real DOM.

How React Uses the Virtual DOM

- React first renders your UI using the Virtual DOM.
- When something changes (like a button click), React **updates the Virtual DOM first**.
- It then compares the updated Virtual DOM with the previous version (this process is called **"diffing"**).
- React finds the exact changes and updates only the affected parts of the Real DOM **(not the whole page)**.

Example

Let's take a look at an example to understand the process.

The Initial Real DOM

index.html (inside the public folder)

```
<!DOCTYPE html>
<html>
  <body>
    <div id="root"></div>
  </body>
</html>
```

This is the Real DOM.

React will target the div with **id="root"** and inject the whole app into it.

Creating the Virtual DOM

App.js (inside src folder)

```
import { useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
    </div>
  );
}
```

React creates a Virtual DOM that looks like this initially:

```
<p>You clicked 0 times</p>
<button>Click me</button>
```

User Interaction Happens

- User clicks the button.
- *setCount(count + 1)* updates the state to 1.

Now React knows:

- **Something changed in the UI.**
- It doesn't directly change the Real DOM.
- **It first updates the Virtual DOM** and reruns the App function to create the new UI structure.

React Diff's the Virtual DOMs

React compares: **(Old Virtual DOM)**

<p>You clicked 0 times</p>

New Virtual DOM:

<p>You clicked 1 times</p>

React sees only one change: the text inside <p>.

So it updates **only that part** in the Real DOM.

How React Updates the Real DOM

After React builds the Virtual DOM, it needs to show it on the actual web page.

To do this, it:

1. **Finds the root element** in the real DOM
2. **Renders the app** inside that element using `ReactDOM.render()`

Index.js

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";

// Step 1: Get the real DOM element with
//          id="root"
const rootElement = document.getElementById("root");

// Step 2: Render the <App /> into that root
//          element
ReactDOM.render(<App />, rootElement);
```

Explanation

- **<App />** is your whole React app (built using JSX).
- rootElement is the **<div id="root">** in index.html.
- **ReactDOM.render()** tells React to inject the virtual DOM tree (created from App) into the real DOM.

So this is the **moment when the virtual DOM is synced with the real DOM** for the first time.

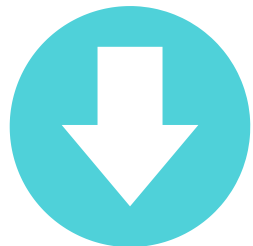
Note

ReactDOM.render() method was used to render the Virtual DOM into the Real DOM in older React versions (**before React 18**).

From React 18 onwards, this got replaced by **createRoot()** but the basic idea is still the same:

Find the root element → render your app inside it.

React



If it was Helpful